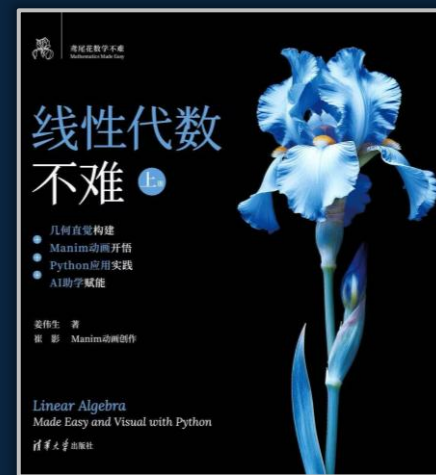


01 向量

1.3 向量加减法

从平行四边形法则到三角形法则

鸢尾花书 · 《线性代数不难》 | 开源代码与书稿: <https://github.com/Visualize-ML>



本节你将掌握的核心技能

1

理解向量加减法的代数定义： n 维向量逐分量相加、相减

2

掌握向量加法的几何直观：平行四边形法则与三角形法则

3

理解并能验证加法的交换律 $\mathbf{a} + \mathbf{b} = \mathbf{b} + \mathbf{a}$ 与结合律

4

掌握平行六面体法则：三个（不共面）向量相加的空间图像

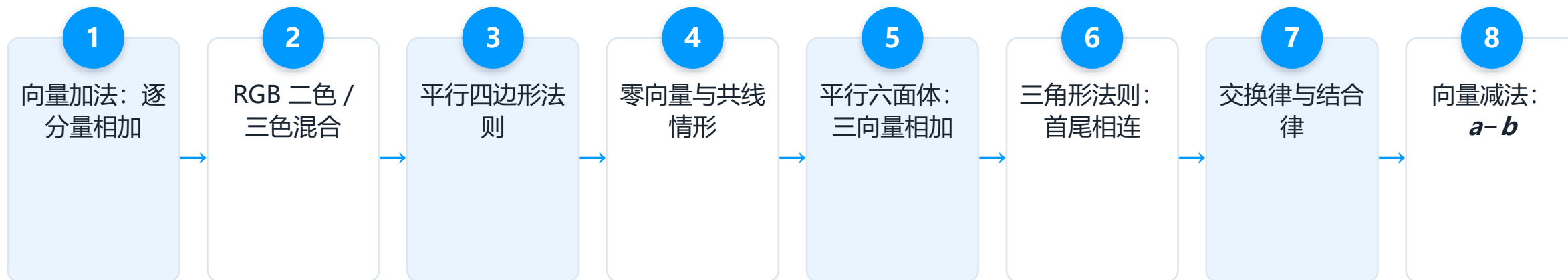
5

理解零向量、反向量，并掌握减法与加法的关系
 $\mathbf{a} - \mathbf{b} = \mathbf{a} + (-\mathbf{b})$

6

会用 NumPy、Matplotlib 编程实现向量加减法及其可视化

本节知识脉络



向量加法：逐分量相加

$$\mathbf{a} + \mathbf{b} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} = \begin{bmatrix} a_1 + b_1 \\ a_2 + b_2 \\ \vdots \\ a_n + b_n \end{bmatrix}$$

- 两个 n 维向量 $\mathbf{a}$ 、 $\mathbf{b}$ 相加，要求维度相同，逐分量对应相加
- 结果仍是一个 n 维向量，这是向量加法最基本、最直接的代数定义
- a_1 、 a_2 、...、 a_n 为向量 $\mathbf{a}$ 的各个分量，均为标量（未知数斜体表示）
- 下一页将用 NumPy 一行代码验证：`a_vec + b_vec`

NumPy 实现向量加法

`np.array()` 创建向量, 直接用 `+` 完成逐分量相加

- 导入 NumPy, 用 `np.array()` 分别创建向量 `a_vec`、`b_vec`
- 直接执行 `a_vec + b_vec`, NumPy 自动完成逐分量相加
- 无需写 for 循环, 向量化运算简洁高效
- 结果仍是一个 NumPy 一维数组, 即新的向量

```
## 初始化
import numpy as np

## 定义向量
a_vec = np.array([4, 1])
b_vec = np.array([1, 2])

## 向量加法
a_plus_b = a_vec + b_vec
```



图1 • 向量加法：混合两种基本色

- e_1 =纯红、 e_2 =纯绿、 e_3 =纯蓝；两两相加得到二次色
- $e_1 + e_2$ =黄色， $e_1 + e_3$ =品红， $e_2 + e_3$ =青色 —— 颜色叠加即向量相加

(a)

$$\begin{array}{c} e_1 \\ \text{red circle} \end{array} + \begin{array}{c} e_2 \\ \text{green circle} \end{array} = \text{yellow circle}$$
$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$$

(b)

$$\begin{array}{c} e_1 \\ \text{red circle} \end{array} + \begin{array}{c} e_3 \\ \text{blue circle} \end{array} = \text{magenta circle}$$
$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

(c)

$$\begin{array}{c} e_2 \\ \text{green circle} \end{array} + \begin{array}{c} e_3 \\ \text{blue circle} \end{array} = \text{cyan circle}$$
$$\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$$

图2 · 向量加法：混合三种基本色

- 三个基色向量同时相加： $\mathbf{e}_1 + \mathbf{e}_2 + \mathbf{e}_3 =$ 白色
- 这是加法从两个向量推广到三个向量的自然延伸
- 本质仍是逐分量相加：
 $[1, 0, 0]^T + [0, 1, 0]^T + [0, 0, 1]^T = [1, 1, 1]^T$
- $[1, 1, 1]^T$ 恰是 RGB 立方体中代表纯白色的顶点

$$\begin{array}{ccccccc} \mathbf{e}_1 & & \mathbf{e}_2 & & \mathbf{e}_3 & & \\ \text{●} & + & \text{●} & + & \text{●} & = & \text{○} \\ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} & + & \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} & + & \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} & = & \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \end{array}$$

平行四边形法则

$\mathbf{a}$ 、 $\mathbf{b}$ 共起点 $\rightarrow$ 以 $\mathbf{a}$ 、 $\mathbf{b}$ 为邻边作平行四边形 $\rightarrow$ 对角线即为 $\mathbf{a}+\mathbf{b}$

- 将 $\mathbf{a}$ 、 $\mathbf{b}$ 平移到同一起点，以两者为邻边补全一个平行四边形
- 从公共起点出发的那条对角线，方向与长度就是和向量 $\mathbf{a}+\mathbf{b}$
- 这是向量加法最经典的几何图像，直观展示“合力”“合速度”等物理意义

图3 · 平行四边形法则的几个例子

- (a)–(c) 单位正方形、正方形、矩形： a 、 b 互相垂直的特殊情形
- (d)–(f) 一般的平行四边形： a 、 b 夹角任意，法则依然成立

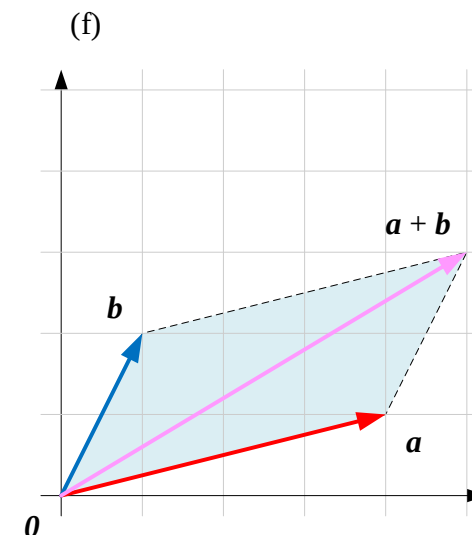
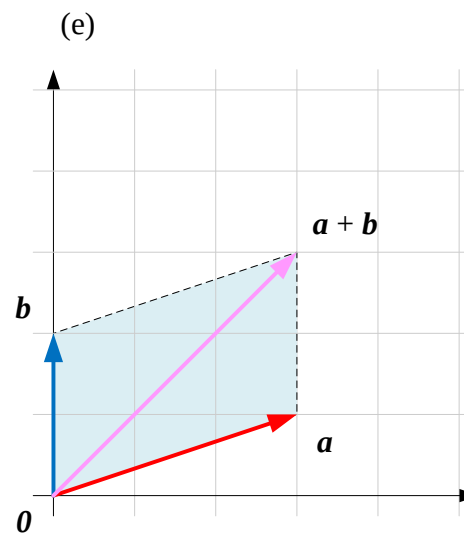
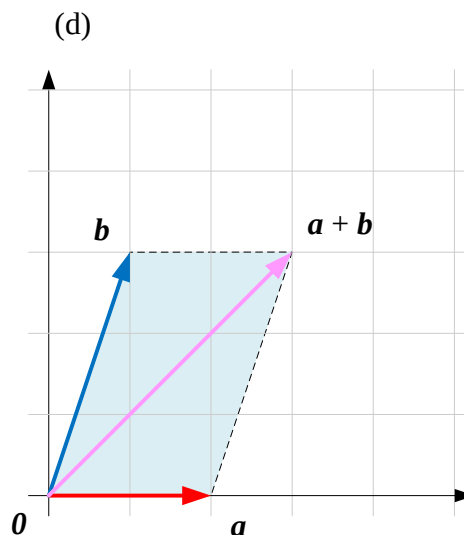
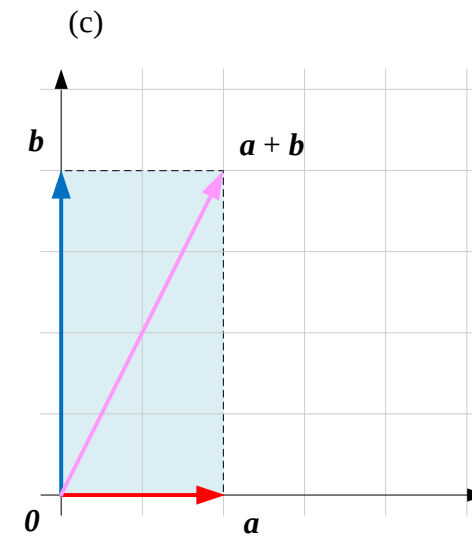
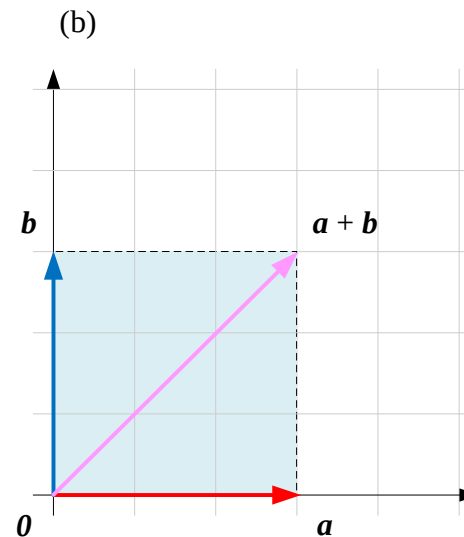
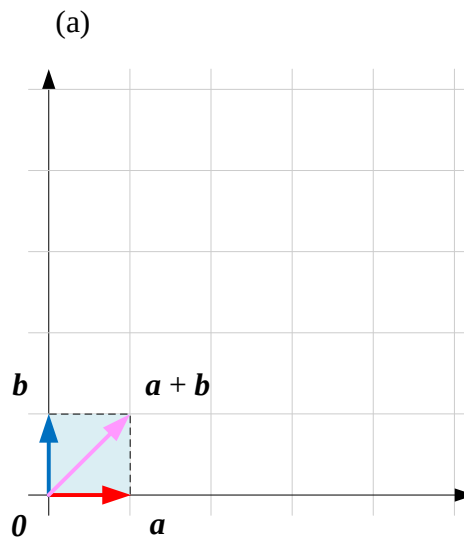
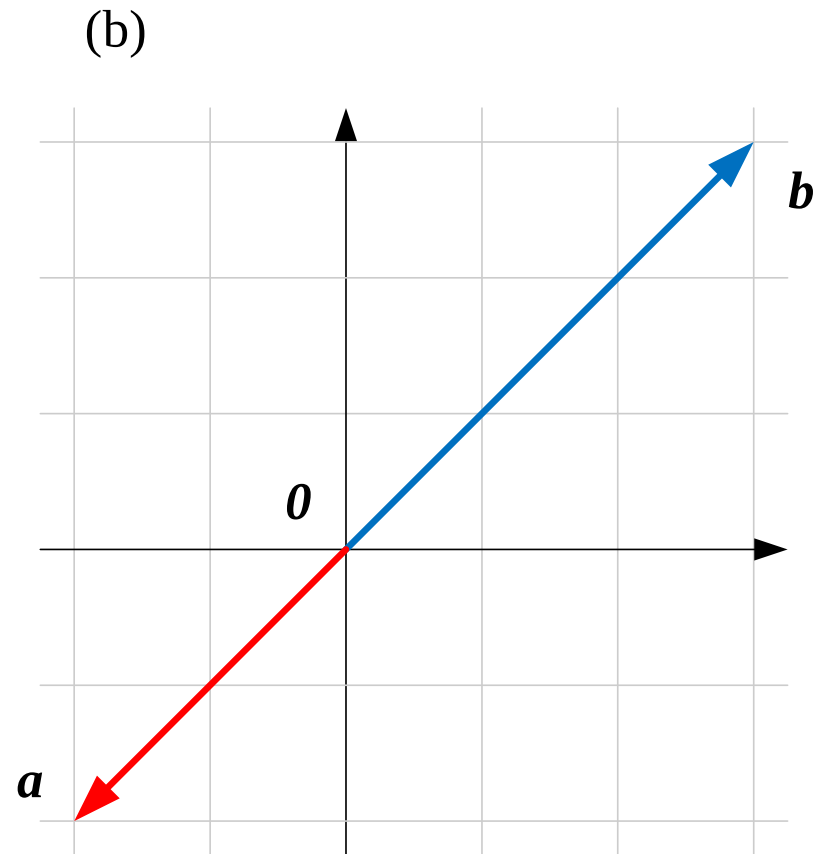
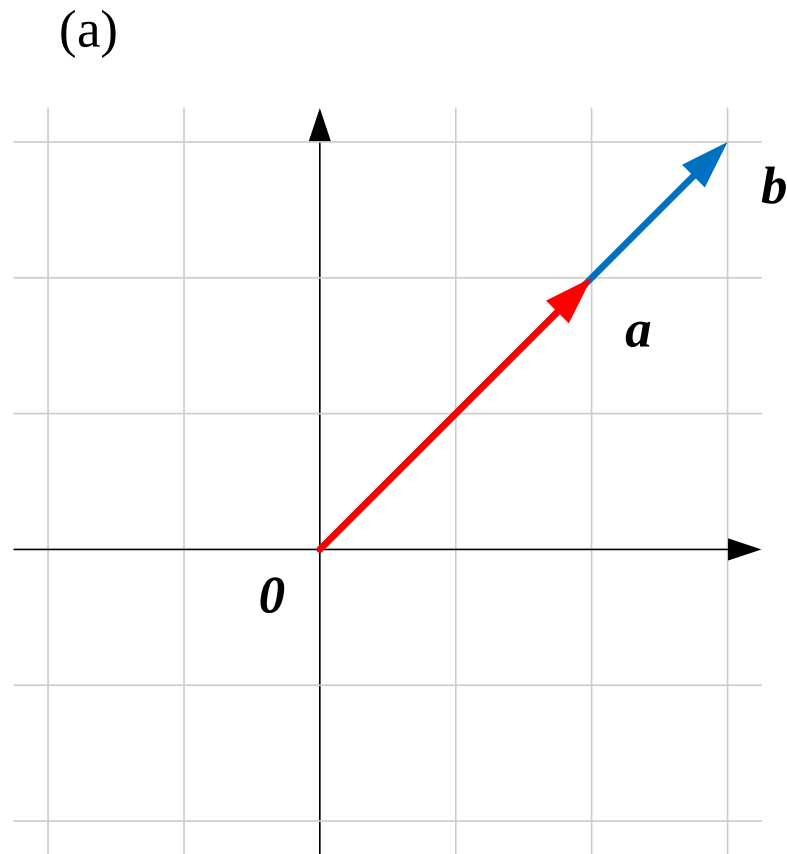


图4 • a 、 b 共线

- 当 a 、 b 方向相同或相反（共线）时，无法补出真正的平行四边形
- 此时“平行四边形”退化为一条线段， $a+b$ 仍可直接逐分量计算
- 特例：若 b 为零向量 0 ，则 $a+0=a$ ，图形上没有位移
- 共线是平行四边形法则的退化情形，而非法则失效



平行四边形法则可视化

`quiver()` 画箭头, `plot()` 补画平行四边形辅助线

- (a)(b) 导入 NumPy、Matplotlib, 定义向量 `a_vec`、`b_vec`
- (c) 计算 `a_plus_b = a_vec + b_vec`
- (d) 创建 6×6 正方形画布 `figsize=(6,6)`
- (e)(f) 用 `quiver()` 分别画红色 `a_vec`、蓝色 `b_vec`
- (g) 用 `quiver()` 画粉色的和向量 `a_plus_b`
- (h)(i) 用 `plot()` 补画两条虚线, 围成平行四边形
- (j) 设置等比例坐标轴、坐标范围、网格与图例

```
## 初始化
import numpy as np
import matplotlib.pyplot as plt

## 定义向量
a_vec = np.array([4, 1])
b_vec = np.array([1, 2])

## 计算向量和
a_plus_b = a_vec + b_vec

## 可视化
plt.figure(figsize=(6,6))

# 绘制向量 a
plt.quiver(0, 0, a_vec[0], a_vec[1],
           angles='xy', scale_units='xy',
           scale=1, color='r', label="a")

# 绘制向量 b
plt.quiver(0, 0, b_vec[0], b_vec[1],
           angles='xy', scale_units='xy',
           scale=1, color='b', label="b")

# 绘制向量 a + b
plt.quiver(0, 0, a_plus_b[0], a_plus_b[1],
           angles='xy', scale_units='xy',
           scale=1, color='pink', label="a + b")

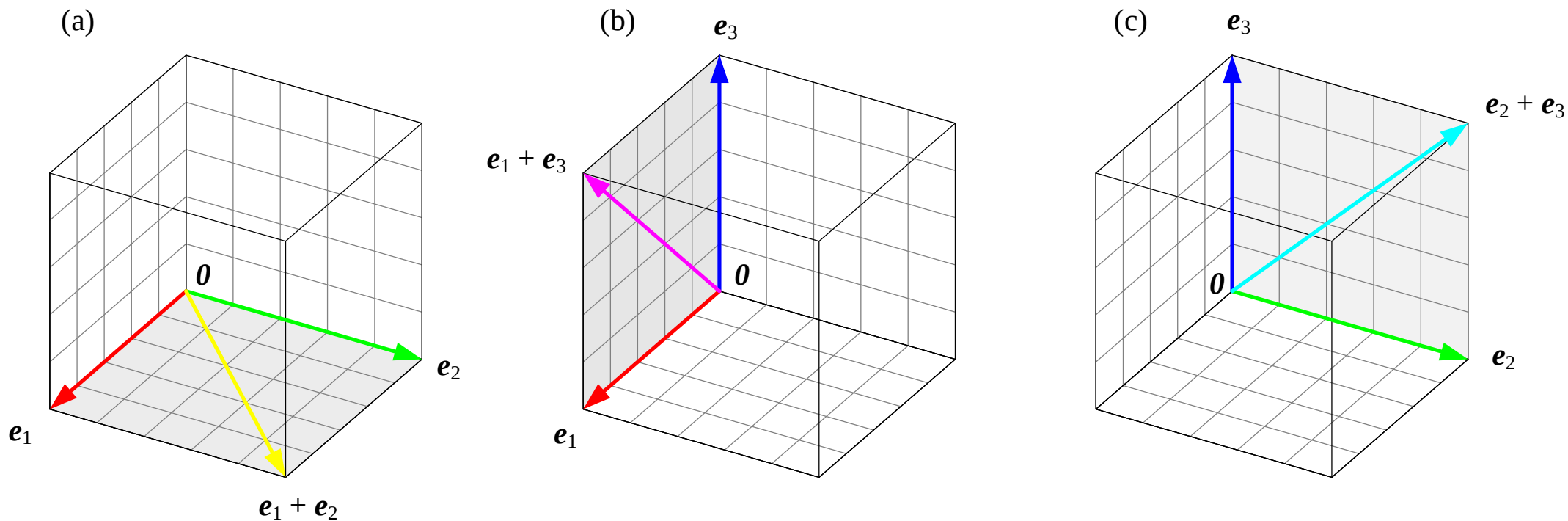
# 画出平行四边形的另外两条边
plt.plot([b_vec[0], a_plus_b[0]],
         [b_vec[1], a_plus_b[1]], 'k--')

plt.plot([a_vec[0], a_plus_b[0]],
         [a_vec[1], a_plus_b[1]], 'k--')

# 装饰
plt.gca().set_aspect('equal', adjustable='box')
plt.xlim(0, 5)
plt.ylim(0, 5)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.grid(color='gray', alpha=0.8, linestyle='-', linewidth=0.25)
plt.legend()
```

图5 • 平行四边形法则：混合两种基本色

- 在三维 RGB 立方体中，同样可以用平行四边形法则理解颜色混合
- $\mathbf{e}_1 + \mathbf{e}_2$ = 黄、 $\mathbf{e}_1 + \mathbf{e}_3$ = 品红、 $\mathbf{e}_2 + \mathbf{e}_3$ = 青；每个二次色都位于以两个基色向量为邻边的平行四边形对角线上
- 把二维法则直接搬到三维空间，几何直观完全一致



零向量: $a + 0 = a$

$$a + 0 = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}$$

- 零向量 0 的每个分量都为 0，是向量加法的“单位元”
- 任意向量加上零向量，结果不变，几何上表示没有位移
- RGB 视角下，零向量 0 对应黑色：无光叠加，仍是黑色
- 黑色逐步叠加 e_1 、 e_2 直至变为黄色，正是加法“累积”的直观例子

平行六面体法则

$\boldsymbol{a}$ 、 $\boldsymbol{b}$ 、 $\boldsymbol{c}$ 不共面 $\rightarrow$ 以三者为棱作平行六面体 $\rightarrow$ 体对角线即为 $\boldsymbol{a}+\boldsymbol{b}+\boldsymbol{c}$

- 三个不共面的向量 $\boldsymbol{a}$ 、 $\boldsymbol{b}$ 、 $\boldsymbol{c}$ 共起点，以三者为棱补全一个平行六面体
- 从公共起点出发的体对角线，就是三者之和 $\boldsymbol{a}+\boldsymbol{b}+\boldsymbol{c}$
- 这是平行四边形法则从二维向三维空间的自然推广

图6 • 用平行六面体理解三个（不共面）向量相加

- (a)–(f) 六组不同的向量 a 、 b 、 c ，均以立方体网格为参照
- 无论三个向量方向如何变化，体对角线始终等于三者之和

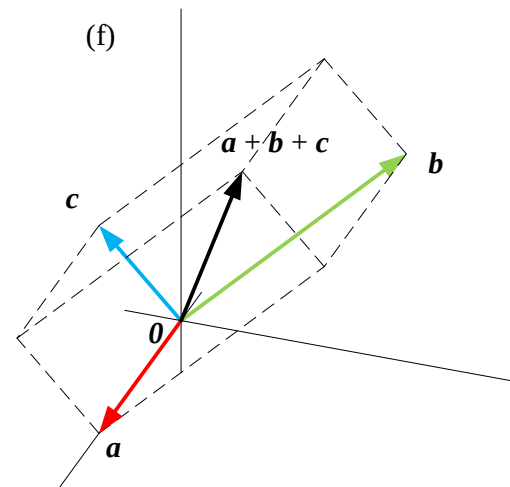
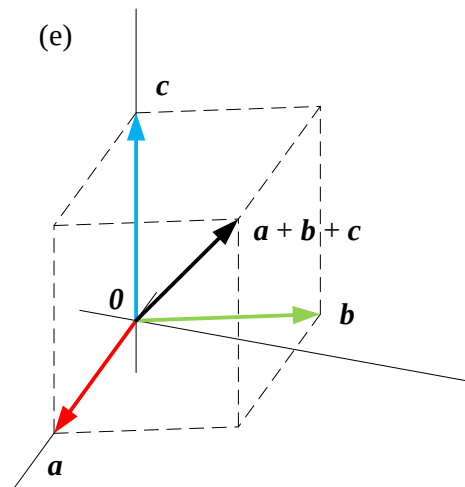
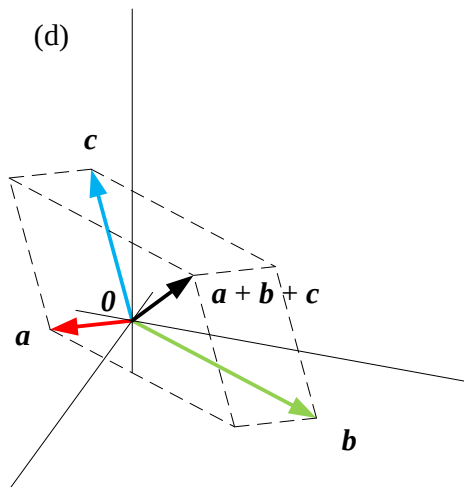
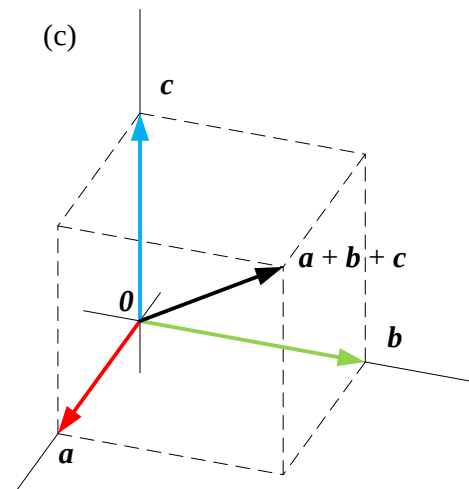
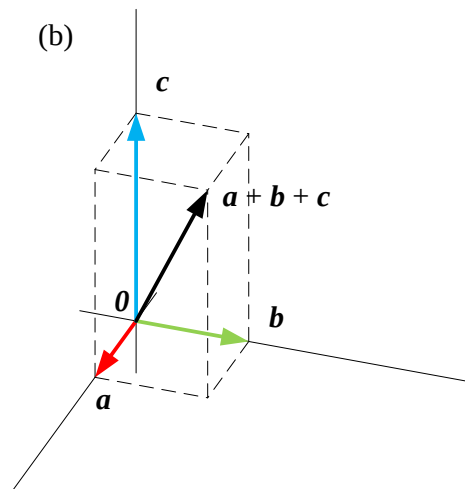
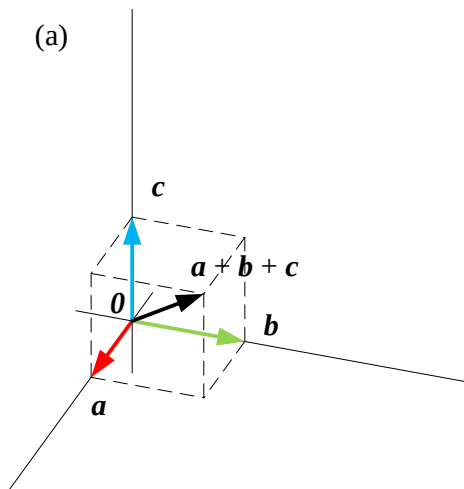


图7 • 三个向量共面

- 当 a 、 b 、 c 共面时，无法构成真正的平行六面体
- (a) 退化为二维的平行四边形，法则本身依然成立
- (b) 若三向量首尾相连恰好围成闭合三角形，则 $a + b + c = 0$
- 闭合三角形的结果，正是下文三角形法则的自然铺垫

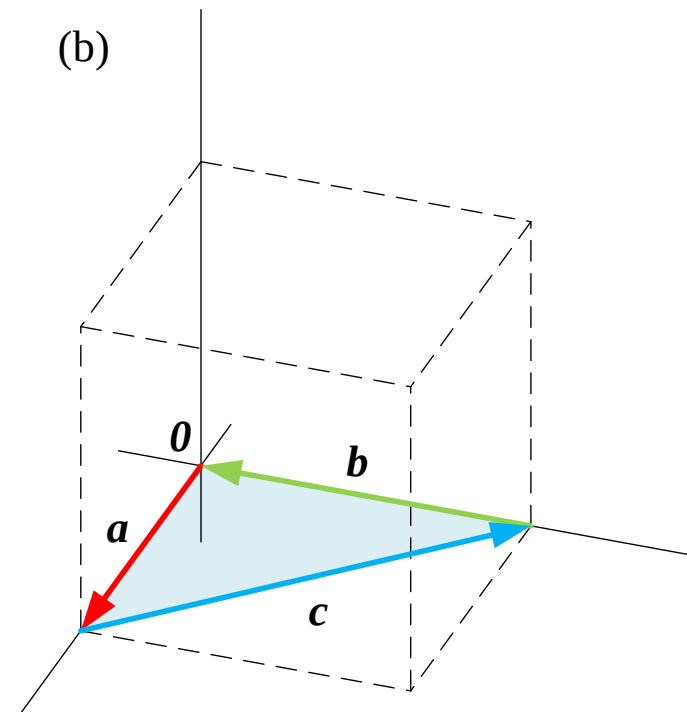
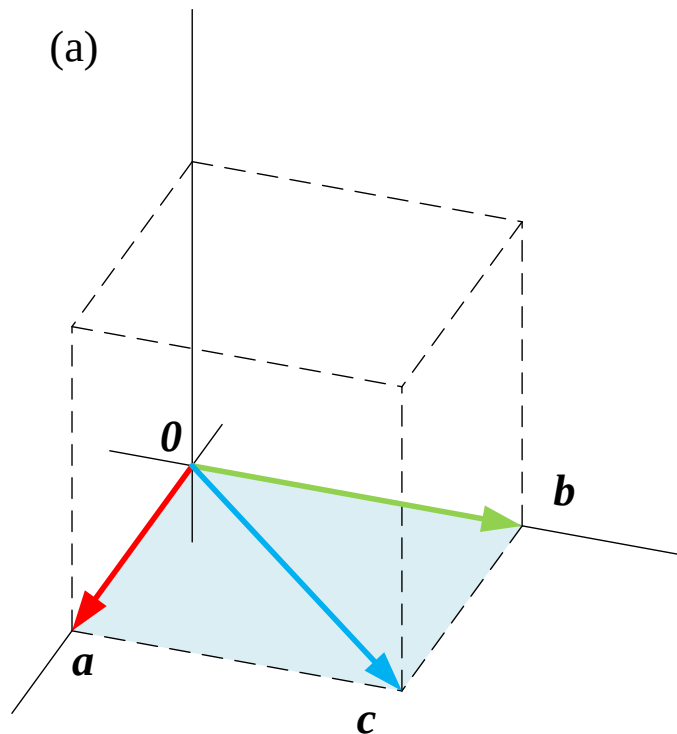


图8 • 平行六面体（单位立方体）计算 $e_1 + e_2 + e_3$ 三者之和

- 以三个基色向量 e_1 、 e_2 、 e_3 为棱，构成单位立方体
- 体对角线的终点 $[1,1,1]^T$ 正是白色所在的位置
- 红、绿、蓝三色等量混合得到白色，几何与颜色直观完全对应

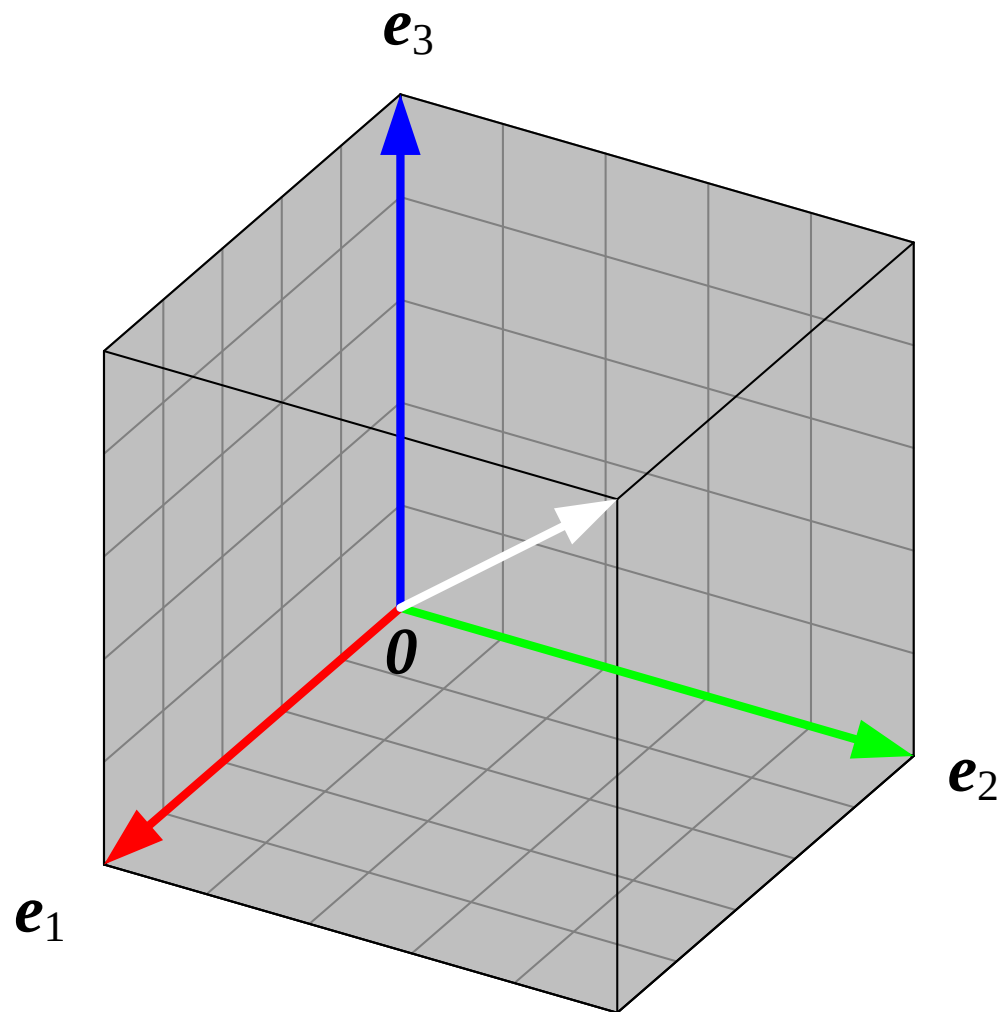
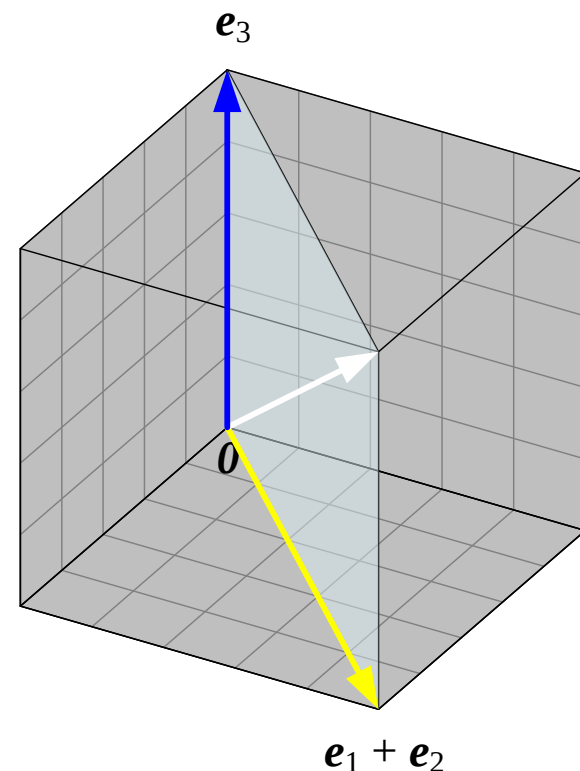
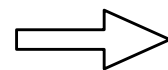
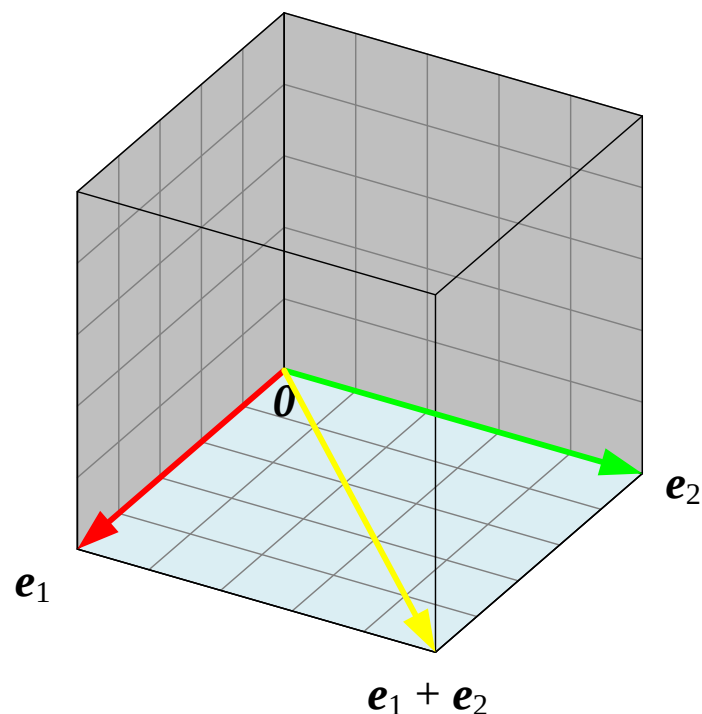


图9 • 拆解成两个平行四边形: $(e_1 + e_2) + e_3$

- 平行六面体也可以拆分成两步平行四边形法则完成
- 第一步: 先求 $e_1 + e_2$ (黄色), 得到底面对角线
- 第二步: 再将结果与 e_3 相加, 得到最终的体对角线
- 这一拆解方式, 为下文的结合律提供了直观铺垫



三角形法则

把 $\mathbf{b}$ 的起点平移到 $\mathbf{a}$ 的终点 $\rightarrow$ 从 $\mathbf{a}$ 的起点指向 $\mathbf{b}$ 的终点, 即为 $\mathbf{a}+\mathbf{b}$

- 将向量 $\mathbf{b}$ 首尾相连地平移, 使其起点与 $\mathbf{a}$ 的终点重合
- 从 $\mathbf{a}$ 的起点出发、指向 $\mathbf{b}$ 终点的箭头, 就是和向量 $\mathbf{a}+\mathbf{b}$
- 三角形法则与平行四边形法则是同一件事的两种画法, 结果完全一致

图10 · 三角形法则计算向量加法 $a+b$ 的几个例子

- (a)–(f) 与图3 完全相同的向量组合，改用三角形法则求解
- 对比图3 可以发现：两种法则得到的和向量位置完全一致

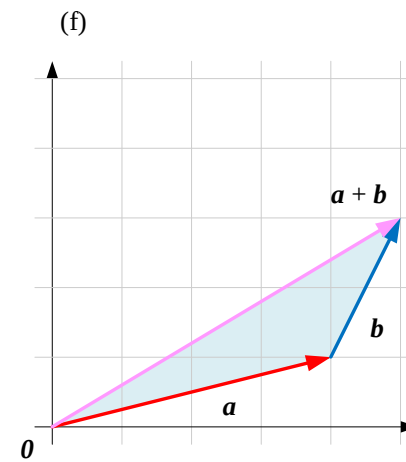
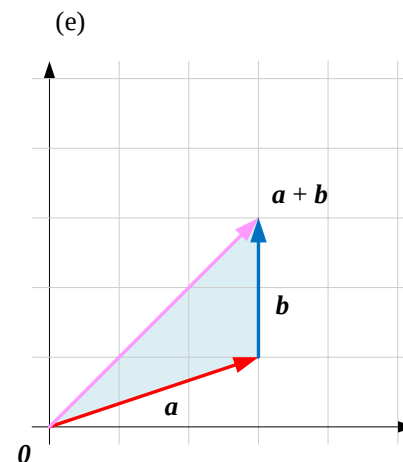
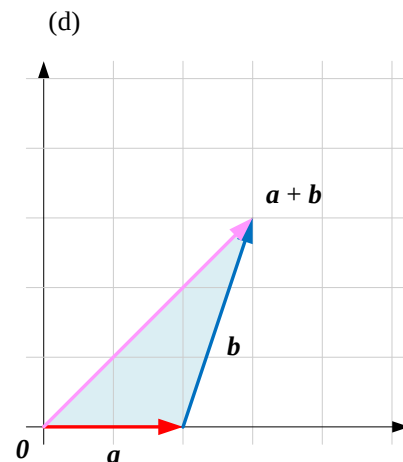
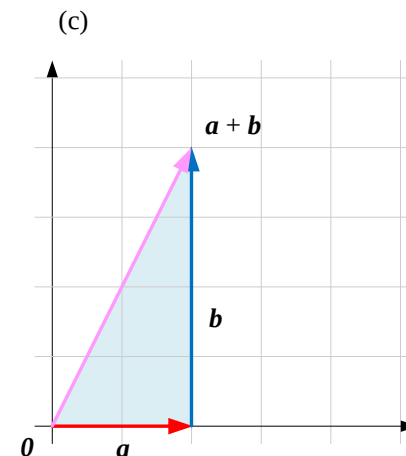
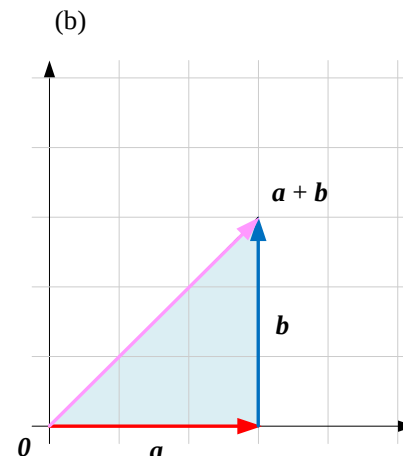
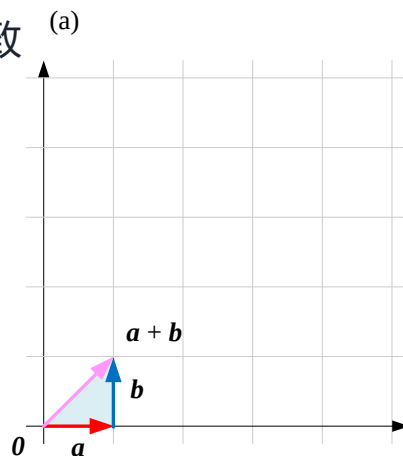
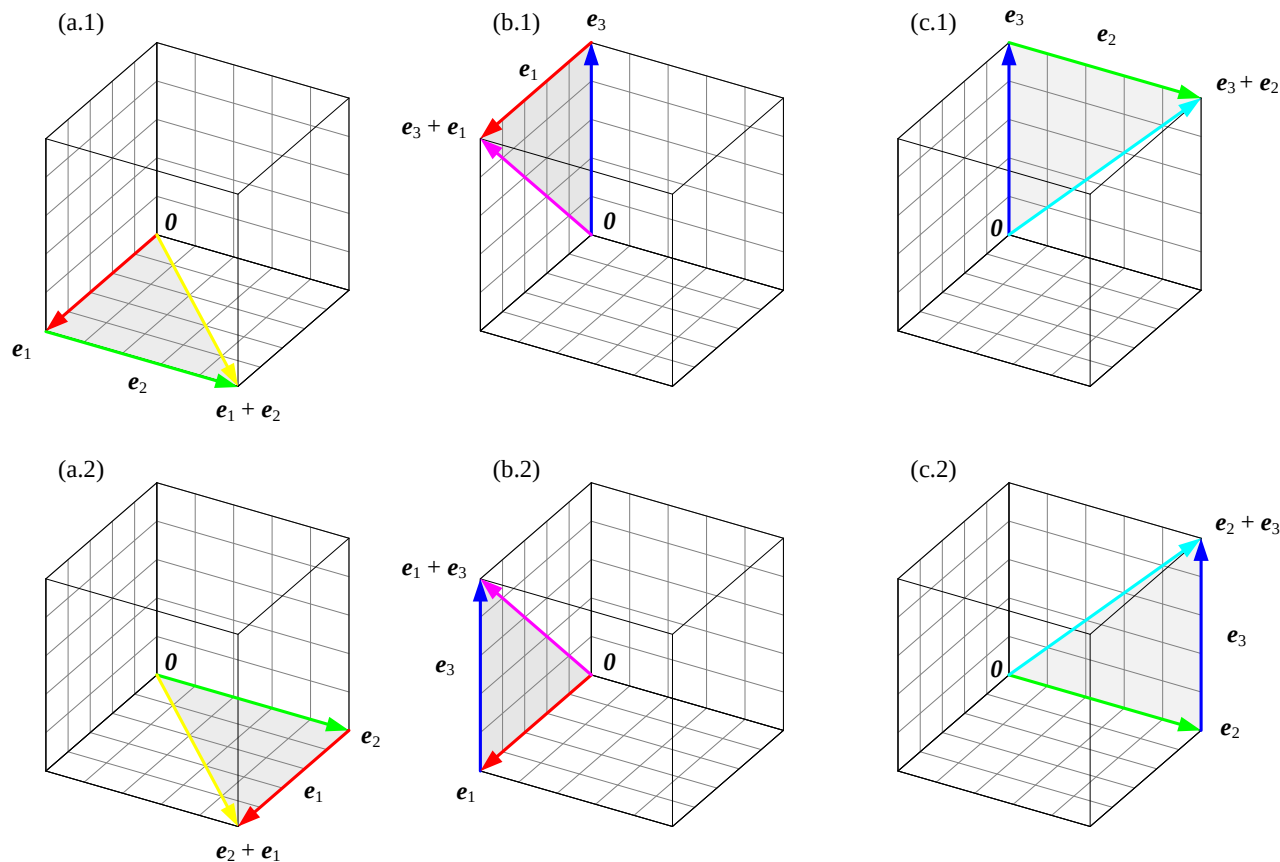


图11 • 向量加法交换律: $e_1 + e_2 = e_2 + e_1$

- a.1/a.2、b.1/b.2、c.1/c.2 分别展示同一对基色的两种相加顺序
- 无论先加哪一个，首尾相连后终点位置不变，颜色结果也完全相同



多个向量相加：首尾顺序连接

$a+b+c =$ 将 a 、 b 、 c 依次首尾相连，起点到终点即为总和

- 三角形法则可以直接推广到任意多个向量依次相加
- 把每个向量依次首尾相连，从第一个向量的起点连到最后一个向量的终点
- 这条连线就是所有向量的和，无需逐个先算平行四边形或平行六面体

图12 • 用首尾相连的三角形法则计算三个向量相加

- (a)–(f) 六组不同的 a 、 b 、 c ，依次首尾相连得到总和
- 与图6 的平行六面体法则相比，三角形法则的画图过程更加直接

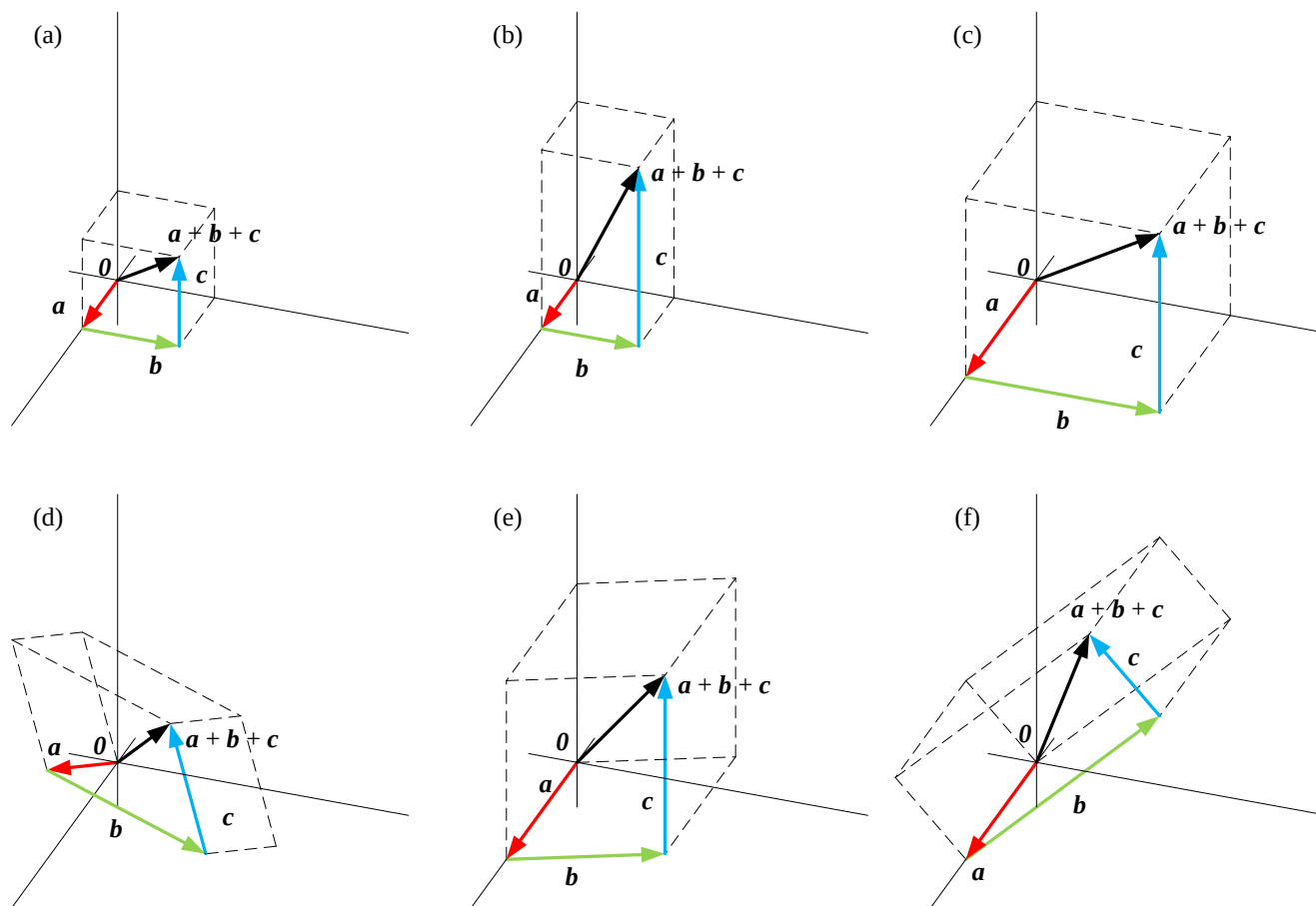
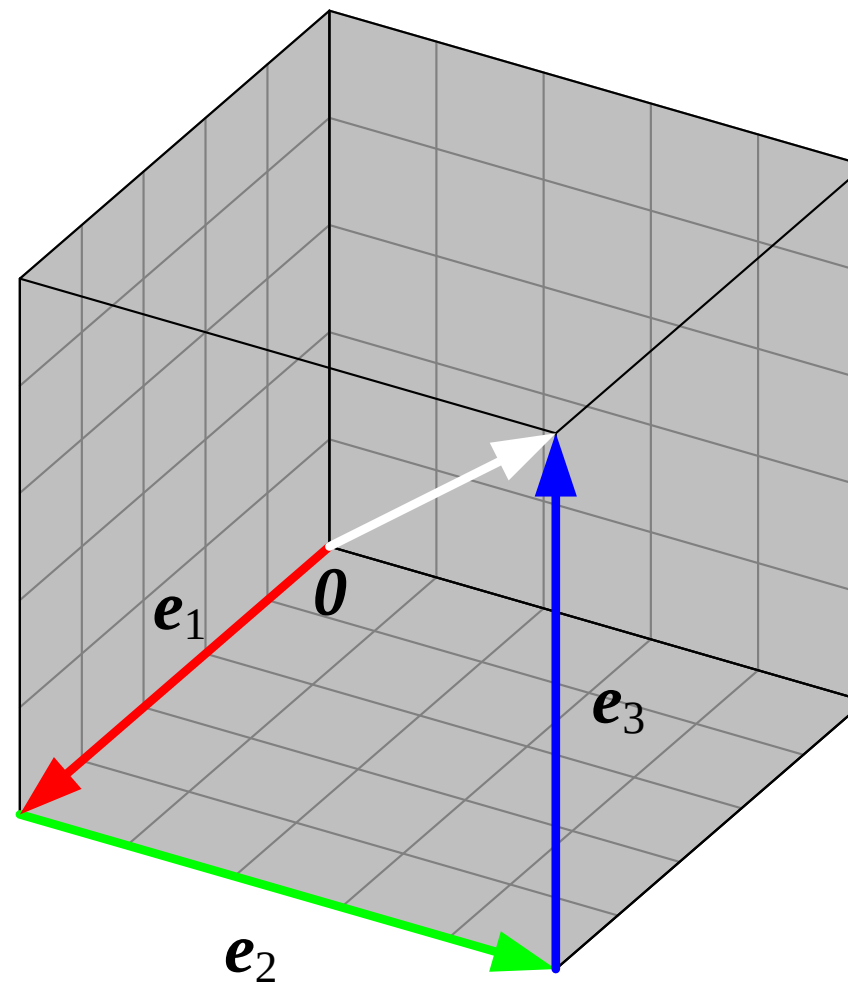


图13 • 首尾顺序连接计算 $e_1 + e_2 + e_3$ 三者之和

- 红、绿、蓝三个基色向量依次首尾相连
- 连线终点同样落在 $[1,1,1]^T$ ，即白色所在位置
- 与图8 的平行六面体法则殊途同归，验证两种几何图像等价



结合律

$$(e_1 + e_2) + e_3 = e_1 + (e_2 + e_3)$$

- 无论先把哪两个向量相加，最终结果都相同——加法满足结合律
- 先算 $e_1 + e_2$ 再加 e_3 ，或先算 $e_2 + e_3$ 再加 e_1 ，殊途同归
- 结合律让“多个向量相加”不必关心运算顺序，只需保证首尾相连的路径正确

图14 • $(e_1 + e_2) + e_3$ 与 $e_1 + (e_2 + e_3)$ 两条路径

- 上排：先求 $e_3 + e_2$ ，再加 e_1 ；下排：先求 $e_2 + e_1$ ，再加 e_3
- 两条完全不同的计算路径，最终都收敛到同一个白色顶点

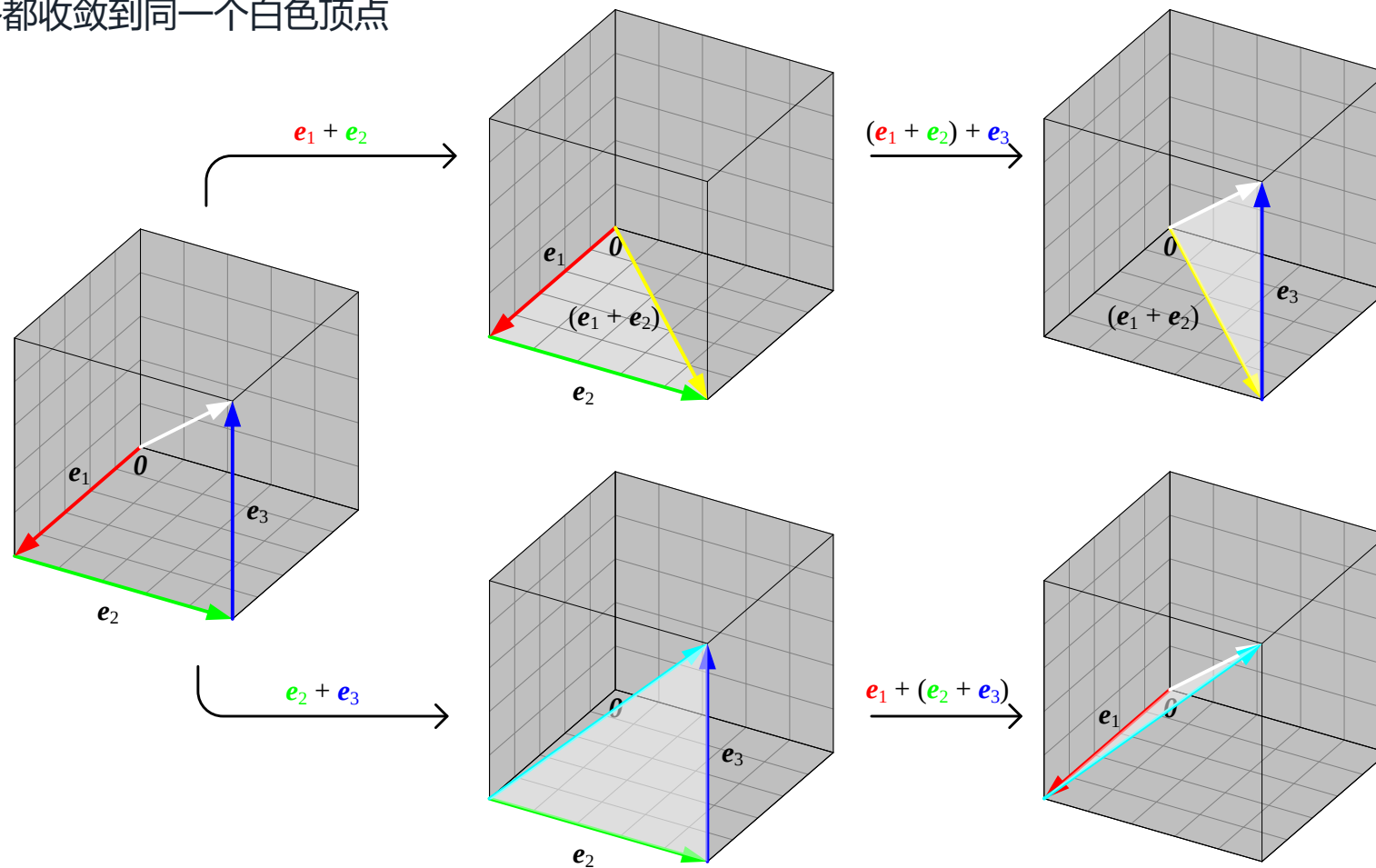
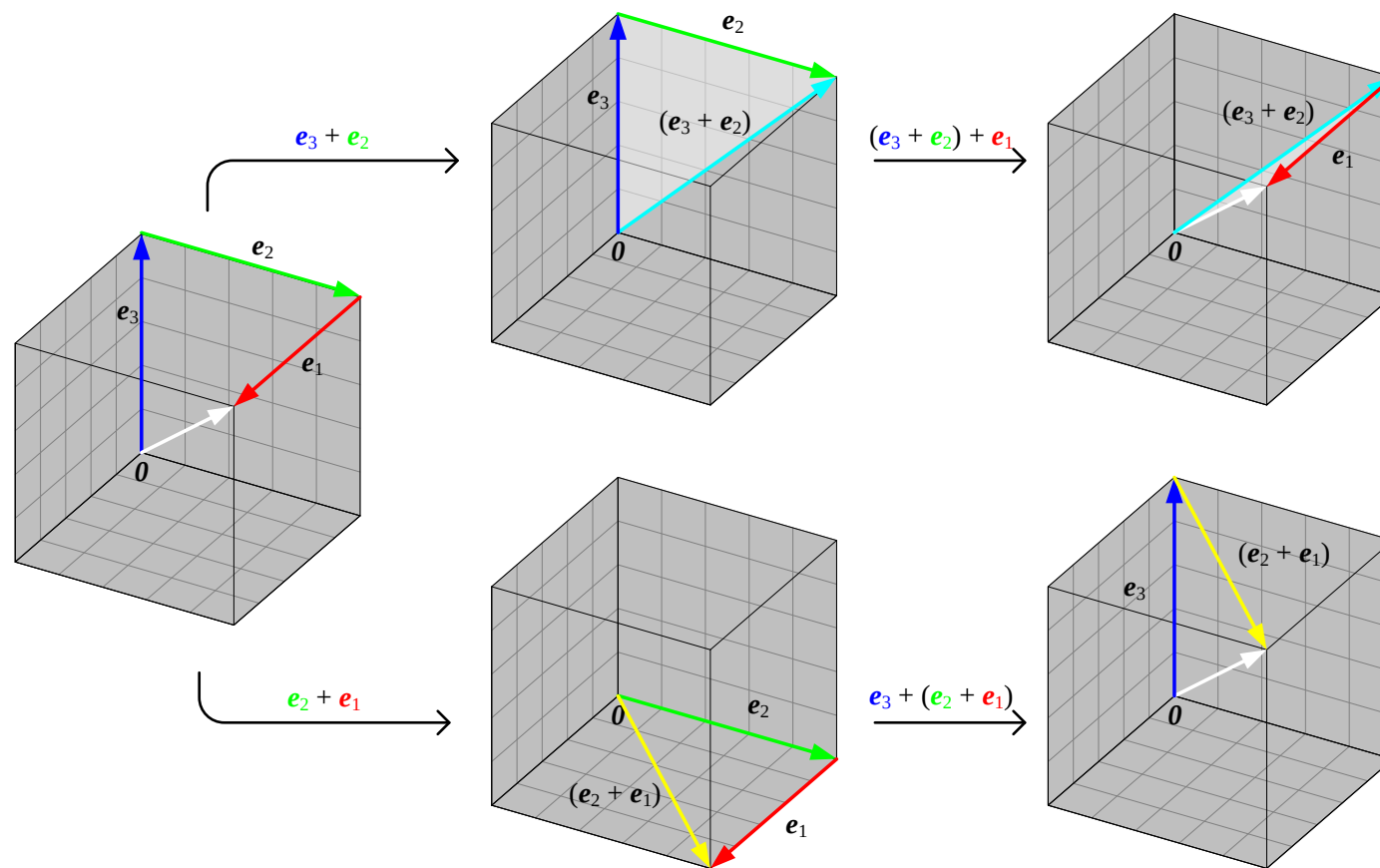


图15 • $(e_3 + e_2) + e_1$ 与 $e_3 + (e_2 + e_1)$ 两条路径

- 调换向量的排列顺序，重新验证结合律，结果依旧收敛到白色顶点
- 思考：红、绿、蓝三个向量还可以按哪些排列组合得到同样的白色？



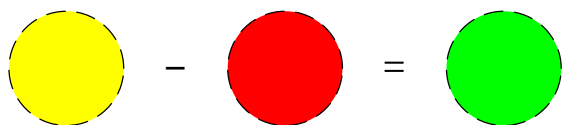
向量减法：逐分量相减

$$\mathbf{a} - \mathbf{b} = [a_1; a_2; \cdots; a_n] - [b_1; b_2; \cdots; b_n] = [a_1 - b_1; a_2 - b_2; \cdots; a_n - b_n]$$

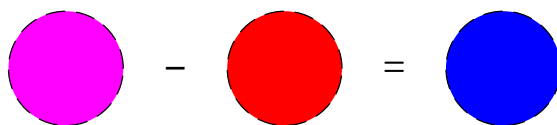
- 两个 n 维向量相减，同样要求维度相同，逐分量对应相减
- RGB 视角下，颜色向量相减，相当于从混合色中“滤除”某种基色
- 例如：黄色（红+绿）减去红色，恰好只剩下绿色分量

图16 • RGB 颜色空间的向量减法

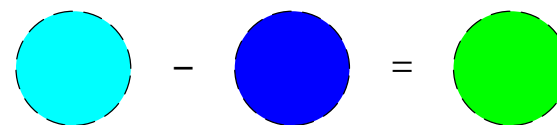
- 黄色-红色=绿色；品红-红色=蓝色；青色-蓝色=绿色
- 颜色相减就是逐分量做减法，直观呈现“滤除某种基色”的效果



$$\begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} - \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$



$$\begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$



$$\begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

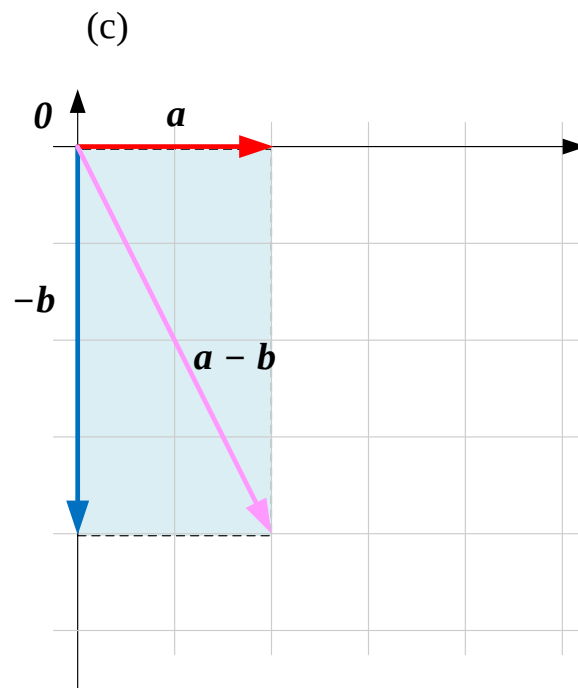
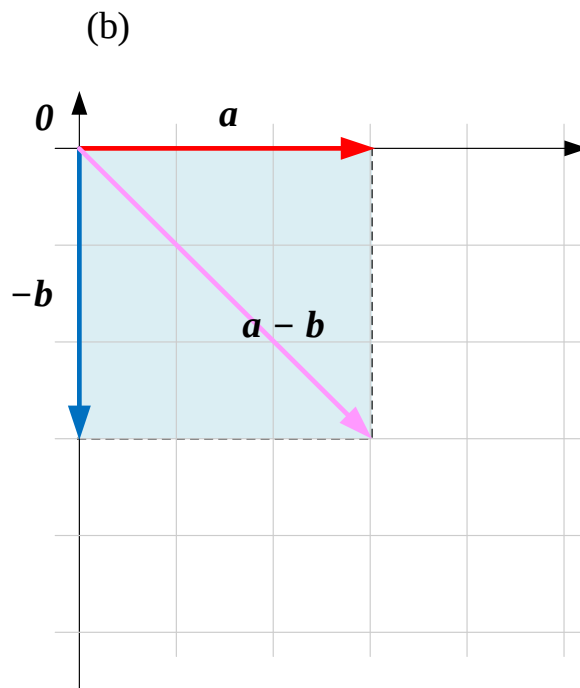
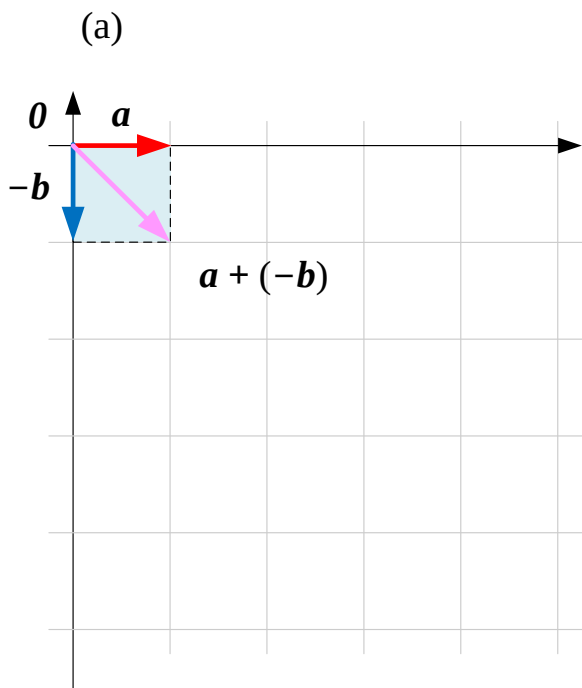
$a - b = a + (-b)$: 反向量

$a - b = a + (-b)$, 其中 $-b$ 为 b 的反向量 (长度相同、方向相反)

- 向量减法可以理解为: 加上被减向量的 “反向量”
- $-b$ 与 b 长度相同、方向相反, 满足 $b + (-b) = 0$
- 这样一来, 减法就转化成了熟悉的加法, 可以直接套用平行四边形/三角形法则

图17 · 平行四边形法则计算向量减法 $a - b$ 的几个例子

- 先画出 $-b$ ，再与 a 用平行四边形法则相加，即得 $a - b$
- (a)–(c) 三组不同的 a 、 b ，展示减法在平面上的几何结果



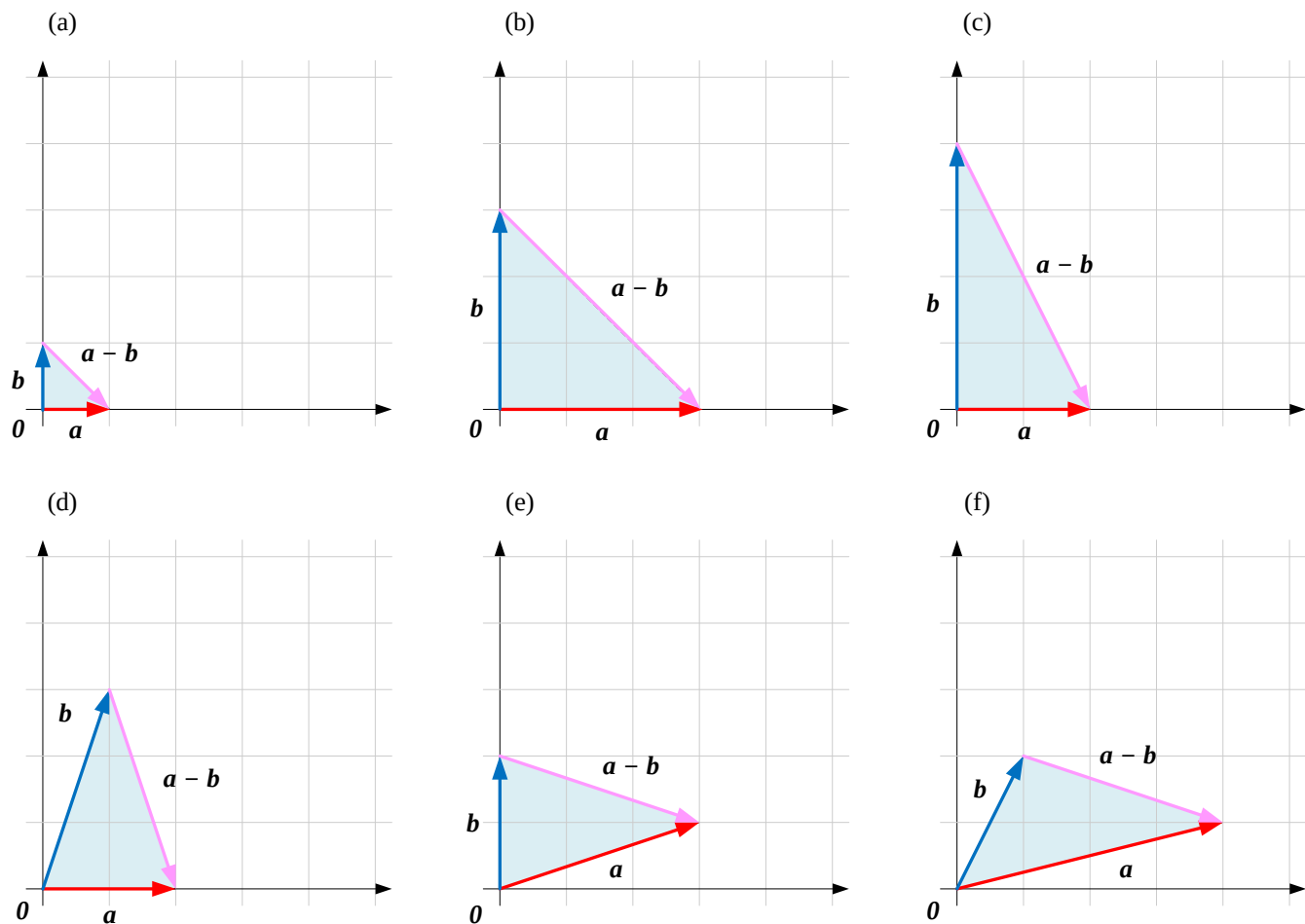
设 $c = a - b$, 则 $b + c = a$

$$c = a - b \Leftrightarrow b + c = a \Leftrightarrow b + (a - b) = a$$

- 若把 c 定义为 $a - b$, 则 b 与 c 相加必须等于 a
- 这正是三角形法则的逆用: 从 b 的终点指向 a 的终点, 就是 $a - b$
- 由此得到减法的另一种几何画法, 无需先求反向量

图18 · 三角形法则计算向量减法 $a - b$ 的几个例子

- (a)–(f) 从 b 的终点指向 a 的终点，箭头即为 $a - b$
- 与图17 的平行四边形法则相比，三角形法则作图更直接、更省步骤



本节关键术语

向量加法 *Vector Addition* — 逐分量相加: $\mathbf{a} + \mathbf{b}$

平行四边形法则 *Parallelogram Law* — 共起点补全平行四边形, 对角线为和

平行六面体法则 *Parallelepiped Rule* — 三个不共面向量相加的空间图像

结合律 *Associative Law* — $(\mathbf{a} + \mathbf{b}) + \mathbf{c} = \mathbf{a} + (\mathbf{b} + \mathbf{c})$

反向量 *Opposite Vector* — $-\mathbf{b}$, 满足 $\mathbf{b} + (-\mathbf{b}) = \mathbf{0}$

向量减法 *Vector Subtraction* — 逐分量相减: $\mathbf{a} - \mathbf{b}$

三角形法则 *Triangle Law* — 首尾相连, 起点到终点为和

交换律 *Commutative Law* — $\mathbf{a} + \mathbf{b} = \mathbf{b} + \mathbf{a}$

零向量 *Zero Vector* — $\mathbf{a} + \mathbf{0} = \mathbf{a}$, 加法单位元

共线 / 共面 *Collinear / Coplanar* — 平行四边形 / 六面体的退化情形

配套代码与延伸阅读

代码1 • LA_01_03_01.ipynb

NumPy 实现向量加法、减法的基础运算

代码2 • LA_01_03_02.ipynb

用 Matplotlib 可视化平行四边形法则

代码3 • LA_01_03_03.ipynb (自学)

用 Matplotlib 可视化三角形法则，巩固首尾相连的画法

开源仓库

全书代码与图像素材: <https://github.com/Visualize-ML>

习题 Q1 – Q10

Q1 请用 `np.array()` 创建向量 $\mathbf{a}=[1,2,3]^T$ 、 $\mathbf{b}=[3,4,5]^T$ 、 $\mathbf{c}=[5,6,7]^T$ ，并计算 $\mathbf{a}+\mathbf{b}+\mathbf{c}$ 。

Q3 请学习 `matplotlib.pyplot.fill()`，修改代码2，为平行四边形填充颜色。

Q5 请编写 Python 代码绘制图8（RGB 平行六面体， $\mathbf{e}_1+\mathbf{e}_2+\mathbf{e}_3$ ）。

Q7 请用平行四边形法则，可视化图3(d)(e)(f)对应的 $\mathbf{a}-\mathbf{b}$ 。

Q9 用 NumPy 随机生成两个 3 维向量，验证加法交换律 $\mathbf{a}+\mathbf{b}=\mathbf{b}+\mathbf{a}$ 。

Q2 请利用代码2可视化图3每个子图 (a)–(f)。

Q4 请修改代码1，计算 $\mathbf{a}-\mathbf{b}$ 、 $\mathbf{b}-\mathbf{a}$ ，其中 $\mathbf{a}=[3,2,1]^T$ 、 $\mathbf{b}=[5,4,3]^T$ 。

Q6 请编写 Python 代码绘制图13（RGB 三角形法则， $\mathbf{e}_1+\mathbf{e}_2+\mathbf{e}_3=\text{白色}$ ）。

Q8 请根据图18，用 Python 绘制 $\mathbf{b}-\mathbf{a}$ 。

Q10 用 NumPy 随机生成三个 3 维向量，验证加法结合律 $(\mathbf{a}+\mathbf{b})+\mathbf{c}=\mathbf{a}+(\mathbf{b}+\mathbf{c})$ 。

习题 Q11 – Q19

Q11 编写函数 `parallelogram(a, b)`，输入两个二维向量，用 Matplotlib 绘制平行四边形法则图。

Q13 用 `quiver()` 同时画出 $\mathbf{a}$ 、 $\mathbf{b}$ 、 $\mathbf{a} + \mathbf{b}$ 、 $\mathbf{a} - \mathbf{b}$ 四个向量，比较方向差异。

Q15 用 NumPy 实现三维平行六面体法则，验证顶点坐标是否等于三个向量之和。

Q17 用 Matplotlib 3D 工具箱绘制 RGB 立方体，并标出 $\mathbf{e}_1 + \mathbf{e}_2 + \mathbf{e}_3$ 的对角线。

Q19 用 Python 验证 $(\mathbf{e}_3 + \mathbf{e}_2) + \mathbf{e}_1$ 与 $\mathbf{e}_3 + (\mathbf{e}_2 + \mathbf{e}_1)$ 是否相等，并思考还有哪些排列顺序。

Q12 编写函数 `triangle(a, b)`，输入两个二维向量，用 Matplotlib 绘制三角形法则图。

Q14 生成 100 组随机二维向量对，统计其中共线（视为退化）向量对所占比例。

Q16 编写代码判断三个向量是否共面（提示：混合积 / 行列式是否为零）。

Q18 编写函数 `vector_chain(vectors)`，用首尾相连的方式可视化任意多个向量之和。

习题 Q20 – Q28

Q20 用 NumPy 实现向量减法 $\mathbf{a} - \mathbf{b}$ ，并验证其等价于 $\mathbf{a} + (-\mathbf{b})$ 。

Q22 用 for 循环（不用向量化）手动实现逐分量加法，与 NumPy 结果对比验证。

Q24 用 Python 生成任意 n 维随机向量 $\mathbf{a}$ ，验证 $\mathbf{a} + \mathbf{0} = \mathbf{a}$ 。

Q26 输入 RGB 三个基色向量的任意 0/1 系数组合，列出所有可能的混合颜色。

Q28 编写函数 `subtract_and_plot(a, b)`，同时用平行四边形法则和三角形法则画出 $\mathbf{a} - \mathbf{b}$ ，并比较两种画法。

Q21 编写代码，给定向量 $\mathbf{a}$ 、 $\mathbf{c}$ ，求满足 $\mathbf{b} + \mathbf{c} = \mathbf{a}$ 的向量 $\mathbf{b}$ 。

Q23 编写函数 `is_zero_vector(v)`，判断输入向量是否为零向量（允许浮点误差）。

Q25 用 Matplotlib 动画（FuncAnimation）演示 $\mathbf{a} + \mathbf{b}$ 从 0 逐渐平移至终点的过程。

Q27 用 NumPy 验证： n 维空间中，任意向量都可以表示为 n 个基向量的加权和。

10 个可深入探讨的 Prompt (1/2)

1 请用生活中的例子解释：为什么向量加法要用“平行四边形法则”，而不是直接把两个箭头的长度相加？

2 请证明二维、三维向量加法满足交换律和结合律，并说明这对更高维向量是否依然成立？

3 请解释为什么“三个共面向量”无法构成一个真正的平行六面体，并用图形辅助说明。

4 如果把 RGB 颜色的加法混合看作向量加法，那么“向量减法”对应现实中的什么现象？

5 请设计一个 Python 小实验，验证任意排列顺序相加多个向量，其和是否保持不变。

10 个可深入探讨的 Prompt (2/2)

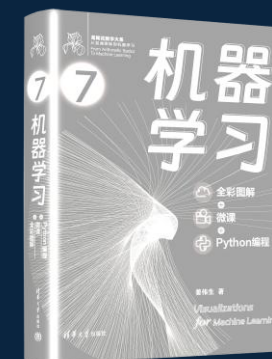
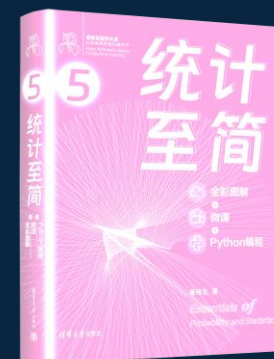
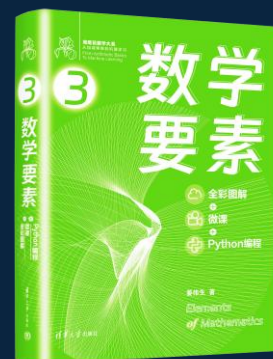
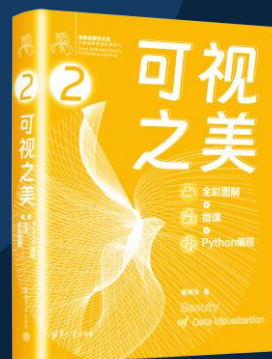
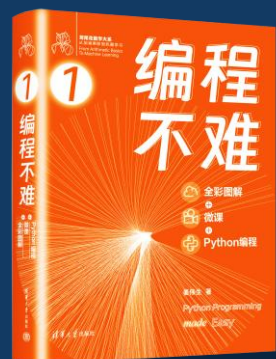
- 6 向量的“反向量” $-b$ 与标量的“相反数” $-k$ 有什么共同点和不同点？
- 7 请解释平行四边形法则与三角形法则本质上是同一件事的两种画法，并说明各自的适用场景。
- 8 如果两个向量共线，平行四边形法则会“退化”，请说明退化后的图形是什么，并给出代数解释。
- 9 请思考：向量加法的“零向量”和矩阵加法的“零矩阵”在概念上有何相似之处？
- 10 请用一个具体的物理场景（如力、位移或速度合成）解释向量加减法的实际意义。

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

